Coverage-Free Fuzzing: LLM-Guided Refinement of Grammar-Based Test Generators

Anonymous Author(s)
Submission under double-anonymous review

***Abstract*—Grammar-based generators keep fuzzing inputs syntactically valid, but they are written once and rarely revised. We ask what a large language model (LLM) contributes when it writes and rewrites such a generator from a formal grammar and black-box parser feedback alone, with no coverage instrumentation. Our pipeline targets the cJSON parser: an LLM proposes Hypothesis generators, every proposal is sandboxed and validated before it runs, and a sanitizer-instrumented harness classifies each input as accepted, rejected, or crashing. Over fifteen seeded runs, the LLM's generators raise acceptance from 58.2% for a static baseline, half of whose inputs are malformed by design, to 97.1%. A pre-registered control locates the source of this gain. With every campaign result withheld from the prompt, the LLM's generators are statistically indistinguishable (92.9%; $p \approx 0.95$, Cliff's $\delta = -0.02$): the gain comes from the model writing a valid generator from the grammar, not from the feedback, and a feedback ablation agrees. Feedback's one measurable effect, in an exploratory analysis, is that fewer proposals fail to run (29 against 52 of 75). The static baseline's valid half alone accepts every input and yields more distinct input shapes than either LLM arm. A pre-registered replication on a second parser, parson, reverses the acceptance effect (95.6% to 92.4%, $p \approx 0.024$): 614 of the 1,063 inputs parson rejected contain constructs that the JSONTestSuite conformance suite says a parser must accept, so the refined generators reached exactly where parson departs from the standard, and the acceptance score counted this as failure. No crash occurred in 25,500 sanitizer-instrumented executions. Black-box counts are cheap to compute but tell an LLM too little to improve a generator; useful feedback must say why inputs fail and which ones matter. We release the pipeline, harnesses, and all campaign data.**

***Index Terms*—fuzzing, grammar-based testing, property-based testing, large language models, black-box testing, software testing**

I. Introduction

Parsers are among the most exposed pieces of code in any system: they are the first code to touch untrusted input, and a memory-safety bug in a parser is directly attacker-controlled. Fuzzing – automatically generating inputs and observing how a target handles them – is the standard defense, but the two dominant families of fuzzer make different tradeoffs around structure and feedback. Coverage-guided, mutation-based fuzzers such as AFL discover interesting inputs by instrumenting the target and rewarding inputs that reach new code, but for a structured format such as JSON, random byte-level mutation of a seed corpus destroys syntactic validity far more often than it discovers it, so most of the fuzzer's budget is spent on inputs the parser rejects in its first few bytes. Grammar-based and property-based approaches invert this: a generator is written directly against a grammar or schema, so every input is syntactically valid (or deliberately near-valid) by construction, and the fuzzer's budget goes toward exploring the target's *structural* handling of the format rather than re-discovering its lexer. Hypothesis, the property-based testing library we build on, is one such approach: a developer composes strategies – generators for values of a given shape – that can mirror a grammar directly.

The gap is that a hand-written grammar-based generator is usually static. It is authored once, and unless a human revisits it, it keeps producing the same distribution of shapes for the rest of a campaign, no matter what the target's actual behavior turns out to be. Coverage-guided fuzzers close an analogous loop by using instrumentation feedback to reward and retain interesting inputs; grammar-based generators typically have nothing equivalent, because the whole appeal of generating from a grammar is that it does not require instrumenting or even recompiling the target. This is not a minor convenience: many realistic targets – vendored third-party libraries pinned at a specific commit, closed-source or license-restricted binaries, embedded or cross-compiled parsers – cannot be rebuilt with coverage instrumentation at all, or the instrumented build would not be representative of what actually ships. A feedback loop for grammar-based fuzzing is only useful in practice if it can run under exactly this constraint: black-box, with no coverage signal available.

This paper asks a narrow, empirical question under that constraint: given only a formal grammar and the feedback a black-box harness can produce – counts of accepted, rejected, and crashing inputs, the number of distinct input shapes, and the number of distinct rejection messages – what does a large language model (LLM) contribute as the author and reviser of a grammar-based generator, and how much of that contribution comes from the feedback? We build a complete pipeline around the cJSON parser (pinned at v1.7.19) and the ANTLR grammars-v4 JSON grammar. A sanitizer-instrumented harness classifies every input as accepted, rejected, or crashing; a refinement loop gives the LLM the grammar and these counts each iteration and asks for a new Hypothesis generator. Because the LLM's output is executable code, every proposal is AST-checked for disallowed imports and calls and must produce well-formed output before it runs against the target.

We evaluate the pipeline with a seeded, fifteen-run comparison against a static baseline; with a pre-registered control arm that withholds every campaign result from the prompt, to separate what the model brings from what the feedback adds; with a pre-registered replication on a second, independent

parser (parson); and with a smaller ablation over which counts the LLM sees. Concretely, this paper contributes:

- A coverage-free pipeline in which an LLM writes and revises grammar-based Hypothesis generators against a black-box, sanitizer-instrumented target, treating every proposal as untrusted code: it is statically sandboxed and behaviorally checked before it runs.
- A controlled answer to what the loop's gain consists of. Over fifteen seeded runs, the LLM's generators raise acceptance on cJSON from 58.2% to 97.1%, but a pre-registered arm without any feedback is statistically indistinguishable (92.9%, $p \approx 0.95$, Cliff's $\delta = -0.02$). The gain comes from the model writing a valid generator from the grammar; the static baseline's valid half alone accepts every input. In an exploratory analysis, feedback's one effect is fewer proposals that fail to run (29 against 52 of 75).
- A pre-registered replication on parson in which the loop significantly *lowers* acceptance (95.6% to 92.4%, $p \approx 0.024$), with every rejected input traced to its cause. 614 of the 1,063 rejected inputs contain constructs a conformance suite [1] says a parser must accept: the generators reached where parson departs from the standard, and the acceptance score counted it as failure. No crash occurred in 25,500 sanitizer-instrumented executions.
- A feedback ablation consistent with the control: counts alone, counts with the number of rejection signatures, and the full summary reach similar acceptance (four usable runs each; descriptive).
- An account of grammar–implementation mismatches under one shared grammar – for example, duplicate object keys accepted by cJSON and rejected by parson – and of what feedback would need to carry to use them: why an input failed, not only how many did.
- Reproducibility tooling for the generated-input stream (a seeding shim over Hypothesis's internal random state, per-run manifests, and hash-identified harness binaries, with a replay check across a toolchain rebuild), and an explicit statement of what it does not cover.

The rest of the paper is organized as follows. Section II situates this work relative to coverage-guided, grammar-based, and LLM-assisted fuzzing. Section III describes the pipeline, the proxy signal, and the sandboxing of LLM-proposed generators in detail. Section IV reports the cJSON comparison and its no-feedback control, the parson replication, and the feedback ablation. Section V discusses limitations, threats to validity, and what the negative result on parson does and does not show. Section VI concludes.

II. Related Work

a) Coverage-guided fuzzing: American Fuzzy Lop [2] and its successor AFL++ [3] popularized mutation-based, coverage-guided fuzzing: a target is instrumented to report which code ran, and the fuzzer retains and mutates inputs that reach new coverage. Manès et al.'s survey [4] gives a taxonomy of this and related approaches. Coverage-guided fuzzers are highly effective when instrumentation is available, but for a structured input format their byte-level mutations mostly produce syntactically invalid inputs that are rejected before any interesting code runs, and none of this family applies when the target cannot be recompiled with instrumentation – the setting this paper addresses.

b) Grammar-based and generation-based fuzzing: Generating inputs directly from a grammar, rather than mutating bytes, is a much older idea: Purdom's sentence generator [5] produces short strings that exercise every grammar production, originally for testing and debugging parsers themselves. More recent tools combine a grammar with coverage feedback: Superion [6] extends a greybox fuzzer with grammar-aware mutation operators so mutated inputs remain syntactically valid, and Nautilus [7] similarly pairs a grammar-based generator with coverage-guided mutation to reach deeper parser states. Both still assume coverage instrumentation is available to decide which grammar-derived inputs to keep; our setting removes that assumption entirely; the only feedback available is the kind a black-box harness can itself produce, so the loop steers the grammar-based *generator*, not a corpus of retained inputs.

c) Property-based testing: QuickCheck [8] introduced property-based testing: a developer writes generators for values of a given shape and a property that should hold for all of them, and the tool searches for a counterexample. Hypothesis [9], which this work builds directly on, is a Python implementation of the same idea, with composable strategies that can mirror a formal grammar. In this line of work the generator is authored once by a human and is not revised in response to how the target under test actually behaves. Vikram et al. [10] show that LLMs can write property-based tests, including their input generators, from API documentation, and separate a test's validity from its usefulness; we ask the corresponding question for a grammar-based generator and a black-box target.

d) LLMs for fuzzing and test generation: LLMs have recently been used to generate or mutate fuzzing inputs directly. TitanFuzz [11] uses an LLM to generate and mutate programs for fuzzing deep-learning library APIs, relying on the LLM's prior exposure to real usage examples of those APIs rather than a formal grammar. Fuzz4All [12] generalizes this to arbitrary languages/systems by using an LLM both to synthesize a distilled input prompt from documentation and examples and to generate and mutate inputs from it. Ackerman and Cybenko [13] have an LLM read a natural-language format specification and generate parser inputs as seeds for a mutation fuzzer. CodaMOSA [14] takes a related but distinct approach for unit-test generation: search-based test generation runs until coverage plateaus, at which point an LLM is asked for example test cases to help the search escape the plateau – i.e., the LLM is invoked *because* coverage feedback is available and has stalled. Closest to our loop, G2Fuzz [15] has an LLM synthesize and mutate input generators for non-textual formats and pairs them with a coverage-guided mutation fuzzer, and ELFuzz [16] evolves generation-based fuzzers with an LLM under coverage guidance. Both revise generators, as we do, but

select them by coverage. Our setting differs from all of these along the same axis: we assume no coverage signal is available at any point, only the parser's own accept/reject decision and sanitizer output, and the LLM's job is not to produce one-off inputs or test cases but to revise a reusable, executable *generator* against a formal grammar it is also given directly, iterating on the same three coverage-free proxies each round.

e) Testing JSON parsers: JSONTestSuite [1] labels test documents as ones a parser must accept, must reject, or may handle either way under RFC 8259 [17], and Möller et al. [18] differentially test JSON parsers across languages and report many semantic discrepancies. Both start from known inputs. We study a generator that must reach such inputs on its own, and Section IV-B shows that the rejections on our second target fall into these suites' categories.

f) Positioning: This paper sits at the intersection of the second and fourth lines above: like Superion and Nautilus, it uses a grammar to keep generated inputs structurally valid; unlike them, it has no coverage instrumentation to fall back on, and unlike TitanFuzz, Fuzz4All, and CodaMOSA, the LLM never sees coverage data, execution traces, or usage examples, and unlike G2Fuzz and ELFuzz, no coverage signal decides which generator survives – only the grammar text and the same three black-box proxies (acceptance rate, structural diversity, rejection signatures) a human maintaining a Hypothesis strategy by hand would have to reason about manually.

III. Methodology

A. Research questions

This paper asks three questions. **RQ1:** on a fixed target, do LLM-written generators refined with black-box, coverage-free feedback raise acceptance relative to a static baseline, and how much of any gain is due to the feedback rather than to the model's ability to write a generator from the grammar alone? **RQ2:** run unmodified except for the harness, does the pipeline carry over to a second, independently implemented parser, and does RQ1's acceptance-rate effect replicate there? **RQ3:** of the three coverage-free proxies exposed to the proposer, does the specific subset shown – counts alone, counts with rejection signatures, or the full signal – change the acceptance the loop reaches?

B. Target, grammar, and harness

The primary target is cJSON [19], pinned at v1.7.19 (commit `c859b25`). The grammar is the ANTLR [20] grammars-v4 JSON grammar [21], vendored unmodified. A thin C harness reads a candidate input from stdin and parses it with `cJSON_ParseWithLengthOpts(..., require_null_terminated=1)`, the library's strictest entry point: it rejects any input with trailing bytes after a valid value, so an "accepted" input is one where the entire buffer is a single grammar-valid value, matching the grammar's `json : value EOF ;` production exactly. The library's more commonly used `cJSON_Parse` entry point does not enforce full-buffer consumption and was deliberately not used, since it would silently accept inputs the grammar does not.

Reconciling the formal grammar with the target's actual behavior surfaced three adaptations, each a genuine mismatch between what the grammar specifies and what the implementation does: (i) the grammar has no token for `NaN`/`Infinity`, matching cJSON's inability to produce or accept them, so generators exclude non-finite floats; (ii) cJSON accepts duplicate object keys, storing every entry in an internal linked list rather than rejecting or de-duplicating, even though the grammar is silent on duplicate keys – this is treated as valid-but-underspecified territory, not a rejection case, and generators deliberately exercise it; (iii) the full-buffer-consumption requirement above is enforced by the harness's choice of entry point, not by the grammar itself. Section IV reports the analogous set of adaptations found for the second target, parson, where two of the three go the opposite direction under the identical grammar.

C. Outcome classification

Every input is executed as a bounded subprocess (5-second default timeout) against a sanitizer-instrumented build (`-fsanitize=address,undefined`). The result is classified, in order: `timeout` if the process does not return within the bound; `signal:<name>` if it is killed by a fatal signal; `crash` if it exits with a non-zero return code or its stderr contains `AddressSanitizer` or `UndefinedBehaviorSanitizer` text; `accepted` if its stdout begins with `status=accepted`; otherwise `rejected`. Timeouts are classified and reported alongside crashes rather than discarded, since an unbounded hang in a parser is a denial-of-service bug in its own right.

D. A coverage-free proxy signal

With no coverage instrumentation, the loop's feedback consists of three signals computed purely from a campaign's classified outcomes:

- **Acceptance rate** – the fraction of inputs classified `accepted`. A generator accepting close to 0% of its own output is testing only the rejection path, not the parser's structural handling of valid input; acceptance tells the loop how much of its output reaches the parser's valid-input logic.
- **Structural diversity** – the count of distinct *structural fingerprints* seen in a campaign. A fingerprint maps each byte of an input to one of six symbols (the six structural characters `{}[]:,` are kept literal, whitespace maps to `_`, ASCII digits map to `#`, the double-quote maps to `"`, and every other byte maps to `x`), concatenates the result, and truncates it to 256 characters. Two inputs share a fingerprint exactly when their first 256 bytes reduce to the same positional shape string; the count of distinct fingerprints across a campaign is used as a coverage-free proxy for how many different grammar shapes a generator has actually produced. The 256-byte truncation makes this proxy a lower bound on true shape diversity for inputs longer than that, since two structurally different inputs that agree on their first 256 bytes collapse to one

fingerprint; this is a deliberate accuracy/cost tradeoff and is stated as a limitation in Section V.

- **Rejection signatures** – the count of distinct parser rejection messages and offsets seen. The proposer receives only this count, never the messages themselves.

These three numbers, plus outcome counts and unique input lengths, are the entire feedback surface exposed to the LLM each iteration, as one JSON object of integers; no rejection message, example input, coverage data, execution trace, or source excerpt is ever included in the prompt.

E. Refinement loop

Each iteration builds a prompt containing the grammar text and a summary of the most recent campaign that produced data (outcome counts, unique input lengths, unique structural fingerprints, unique rejection signatures), and sends it to the LLM proposer. The first iteration has no campaign yet, so its summary is all zeros, and the prompt never includes a previous generator's source: each iteration writes a new generator from the grammar and the numbers. When a proposal fails, the next prompt also carries its error text. The proposer must return a single Python function, `generated_json`, decorated with Hypothesis's `@st.composite` and built from `st.*` combinators, so that recursive structure is expressed the same way a human-authored Hypothesis strategy would express it. Every reported run uses `gpt-4.1-mini` at temperature 0.2 as the proposer.

F. Sandboxing untrusted, LLM-authored code

Because the proposer's output is executable Python that this pipeline runs locally, every proposal passes through a validator (`load_strategy`) before it ever touches the target binary. The validator first parses the proposal's AST and rejects it outright if it imports anything other than `hypothesis.strategies`, or if it calls `eval`, `exec`, `open`, `compile`, or `__import__` by name. The proposal is then executed with an otherwise ordinary Python builtin namespace, minus a short, explicit blocklist (`eval`, `exec`, `compile`, `__import__`, `open`, `input`, `breakpoint`, `exit`, `quit`, `help`, `globals`, `locals`, `vars`, `getattr`, `setattr`, `delattr`) – ordinary helpers such as `ord`, `chr`, `str`, `len`, and `range` remain available, so a proposal is free to use normal Python rather than being restricted to Hypothesis combinators alone. Finally, the resulting `generated_json` must be callable and must produce a `bytes` value via `.example()`; a proposal that fails any of these checks is rejected and logged, and never reaches the harness. This is deliberately scoped as a filter against *accidental* misuse of untrusted, LLM-authored code, not as a hardened security boundary: Python's object model can reach functionality beyond any blocklisted name (for instance, by walking `().__class__.__base__.__subclasses__()`), and closing that off would require process- or container-level isolation rather than name filtering. We report this limitation directly rather than implying a guarantee the design does not provide.

G. Crash triage and minimization

A sanitizer report is reduced to its `SUMMARY:` line and stack frames with addresses and source line numbers normalized out, then hashed into a 16-character signature; records sharing a signature are treated as one root cause. For each unique signature, Hypothesis's `find` is used to shrink a triggering input to a minimal byte string that still reproduces the same crash or timeout classification – rather than simply keeping the first crash observed – and the minimized input is re-run once, standalone, against the pinned build to verify it reproduces outside the shrinking process itself.

H. Reproducibility engineering

Hypothesis's `.example()` draws from a private internal random state that Python's own `random.seed()` does not reach, and separately orders its candidate batch with the module-level `random.shuffle`. Seeding only one of the two leaves the produced example stream non-reproducible across process invocations; a shim (`seeding.seed_everything`) seeds both, which we verified gives byte-identical example streams across repeated processes on Hypothesis 6.165.5 / CPython 3.14.7. Two documented, non-internal alternatives – `settings(derandomize=True)` and `global_force_seed` – were rejected because both collapse a run's draws into repeats of a single small batch, which would change the generation algorithm itself rather than only fix its starting point. Because this shim depends on an internal implementation detail, the Hypothesis version is pinned in `requirements.txt` and the shim fails loudly rather than silently if the relied-upon attribute is renamed or removed in a future release. Every run additionally persists a manifest (seed, package versions, platform, model, feedback mode, CLI arguments) and a reproducibility report (compiler, harness SHA-256, exact command line, repository commit); the compiled harness binary itself is gitignored and host-compiled, so it is identified by content hash rather than assumed identical across machines. This reproducibility guarantee is scoped to the generated input stream only: it does not extend to LLM proposals themselves (temperature 0.2, no server-side seed), to wall-clock-dependent timeout classification, or to the harness binary's bit-for-bit identity across hosts and toolchains.

I. Experimental design

The static baseline alternates two hand-written Hypothesis strategies. The first draws a bounded JSON value (null, booleans, integers, finite floats, strings, and nested arrays and objects with at most 32 leaves) and serializes it with Python's `json.dumps`. The second appends one suffix to such a document: a comma, a closing bracket or brace, the word `␣trailing`, or a NUL byte. Half of the baseline's inputs are thus malformed by design, to exercise rejection paths. On cJSON, whose full-buffer entry point accepts only the NUL suffix, this bounds the baseline's acceptance near 60%. For RQ1, we run fifteen independent, seeded trials (seeds 0–14)

of this static baseline generator and fifteen independent, seeded trials of the LLM-refined loop against the same pinned cJSON build, each trial consisting of up to five refinement iterations of up to 500 examples. We report acceptance rate, structural fingerprints, and rejection signatures per trial. We compare the arms statistically only on acceptance rate, the scale-free outcome; the count-based metrics are retained as descriptive feedback signals because the two arms execute different total numbers of inputs. Two further arms, whose protocol and decision rules were committed before they ran, separate the model from the feedback. In the *no-feedback* arm (seeds 0–14), every iteration receives the prompt the refined arm's first iteration received – the grammar and an all-zero summary, with no previous error – so no campaign result ever reaches the LLM; all fifteen committed first prompts of the refined arm are byte-identical to it. Its primary test is the refined arm against it, with "no detectable effect" claimed only if $p \geq 0.05$ and $|\delta| < 0.33$. The *valid-half* baseline draws every input from the static baseline's grammar-valid strategy. Because the original harness binary no longer loaded after a toolchain upgrade, both arms ran on a rebuild (Homebrew clang 23.1), which replayed all 7,500 seeded baseline inputs with identical outcomes. For RQ2, we repeat RQ1's design on parson exactly – fifteen seeded runs per arm, seeds 0–14, the same baseline, proposer, budget, and feedback – with only the harness changed. The protocol, the primary test (exact two-sided Mann–Whitney U on per-run acceptance rate), and the reporting rules were committed to the artifact before any run. An earlier exploratory pilot on parson, whose proposals were written outside the loop, is reported separately (Section IV-C). For RQ3, we repeat the RQ1 refined-arm recipe under each of the three feedback modes in Section III-D (`counts`, `counts+rejections`, `full`), five seeded trials (seeds 0–4) per mode against the same cJSON build. In one trial per mode, every one of the five iterations produced an LLM proposal that failed validation, leaving that trial with no executed inputs; following the same `iterations_with_data` convention used throughout, these three trials are excluded, leaving four usable trials per mode.

IV. Evaluation

A. RQ1: cJSON, and what the feedback adds

Table I and Fig. 1 report the four cJSON arms, fifteen seeded runs each. The refined loop raises acceptance from the static baseline's 58.2% to 97.1%, and every refined run exceeds every baseline run: complete rank separation, exact two-sided $p = 2/\binom{30}{15} \approx 1.29 \times 10^{-8}$, the smallest value a design with n=15 per arm can produce. Acceptance is a rate, so this comparison holds although the arms execute different numbers of inputs.

The gain does not come from the feedback. The no-feedback arm receives, in every iteration, exactly the prompt the refined arm received before any campaign had run. Its generators are statistically indistinguishable from the refined arm's ($p \approx 0.95$, Cliff's $\delta = -0.02$), which meets the pre-registered rule for "no detectable effect" ($p \geq 0.05$ and $|\delta| < 0.33$). Its lower mean comes from one run whose only working generator emitted 193 strings that could not be encoded (lone surrogates), which count as not accepted; one run produced no working generator at all. The acceptance gain over the static baseline is therefore the model's: given the grammar, gpt-4.1-mini writes a generator whose output is almost always valid, in the first iteration as in the last. Consistent with this, the refined arm's first-iteration generators, written before any feedback existed, already reach 96.6–100% acceptance where they ran.

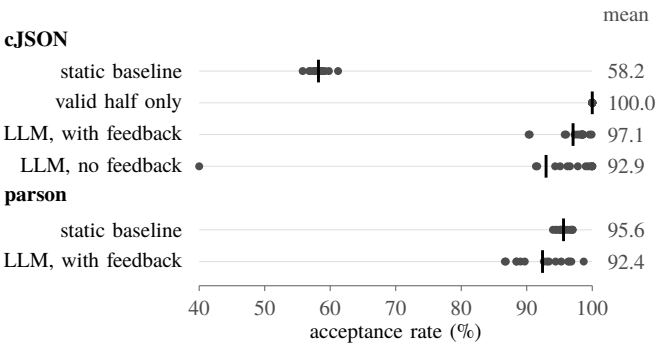


Fig. 1. Per-run acceptance rate of every arm (dots: runs; bar and number: mean). The LLM's generators reach the same acceptance with or without feedback; the static baseline's valid half alone accepts every input.

What the feedback does change. In an exploratory comparison, not part of the pre-registered analysis, the refined arm lost fewer proposals to validation failures (29 against 52 of 75; per run, exact two-sided $p \approx 0.0005$). Its prompts carry both the campaign counts and the previous proposal's error text, so this design cannot separate the two, but the failures are code errors – mostly calls the Hypothesis API rejects – and the error text is the plausible reason the next proposal avoids them. The feedback thus helps the loop keep a working generator; it does not change what working generators produce.

What the baseline comparison measures. Half of the static baseline's inputs carry a deliberate trailing suffix, and cJSON's full-buffer entry point accepts only the NUL suffix, which bounds the baseline near 60%. Its valid half alone accepts every input, and it produces more distinct shapes per 500 inputs (251.0) than the generators of either LLM arm (195.1 and 174.0). The LLM's generators are thus valid but not more varied, and they rarely exercise the rejection path: six of the fifteen refined runs and eight of the fourteen no-feedback runs produced no rejected input at all.

No crashes in any arm. No sanitizer crash, timeout, or fatal signal occurred in any of the sixty runs. The effects measured here concern generator validity, not vulnerability discovery; we do not claim that refinement found, or was likely to find, a memory-safety bug in cJSON.

B. RQ2: replication on parson

We reused every pipeline component unchanged against a second, independent target, parson [22] (commit `ba29f4e`,

TABLE I

cJSON, fifteen seeded runs per arm (mean over runs with data). Shapes are distinct structural fingerprints per 500-input iteration, so arms with different budgets compare directly. Exact two-sided Mann–Whitney U on per-run acceptance: refined vs. static $p \approx 1.29 \times 10^{-8}$ ($\delta = +1.00$); no feedback vs. static $p \approx 1.8 \times 10^{-5}$ ($\delta = +0.86$); refined vs. no feedback $p \approx 0.95$ ($\delta = -0.02$).

Arm	Acceptance (sd)	Rejected inputs	Shapes per 500	Inputs per run	Failed proposals
Static baseline	58.21% (1.30)	41.79%	274.5	500	–
Valid half of the baseline only	100.00% (0.00)	0.00%	251.0	500	–
LLM, with feedback (refined)	97.07% (2.93)	1.95%	195.1	1,533	29 of 75
LLM, no feedback (14 of 15 runs with data)	92.95% (15.53)	3.23%	174.0	821	52 of 75

TABLE II

parson: baseline vs. LLM-refined, fifteen seeded runs per arm, the same design as RQ1's refined and static arms. p: exact two-sided Mann–Whitney U on per-run acceptance rate.

Metric	Baseline	Refined
Acceptance rate, mean (sd)	95.61% (0.95)	92.41% (3.96)
Acceptance rate, per-run range	94.0–97.0%	86.7–98.7%
Crashes / timeouts / signals	0	0
Distinct rejection signatures	1	1
Refined − baseline: -3.20 pp, $p \approx 0.024$, $\delta = -0.48$		

v1.5.3); only the harness differs. Reconciling the same JSON grammar against parson's real behavior surfaced three adaptations, two of which go the *opposite* direction from the cJSON adaptations reported in Section III under the identical grammar: (i) unlike the cJSON harness, parson's parse entry point does not require full-buffer consumption, so trailing garbage after a valid value is silently accepted – a superset of what the grammar's `json : value EOF ;` production specifies; (ii) parson explicitly rejects any object containing a duplicate key, propagating the rejection up as a failure for the whole object – a subset of what cJSON accepts, even though the grammar itself is silent on duplicate keys for both; (iii) parson enforces a fixed nesting limit (`MAX_NESTING = 2048`), rejecting deeper structures outright rather than exhibiting unbounded recursion. That the same formal grammar under-determines a directly opposite validity decision (duplicate keys: accept vs. reject) on two independently implemented parsers is, on its own, evidence that a purely formal grammar specification cannot supply the target-specific signal needed to fuzz either implementation precisely.

Replication. Table II reports the pre-registered replication of RQ1's design on parson. Refinement *lowered* acceptance: 92.4% against the baseline's 95.6% (exact two-sided Mann–Whitney $p \approx 0.024$, Cliff's $\delta = -0.48$). The baseline has tied values, which the exact test assumes away; a tie-corrected test ($p \approx 0.025$) and a permutation test on the difference in means ($p \approx 0.005$) agree. No crash, timeout, or signal occurred in 25,500 sanitizer-instrumented executions, and in 39 of the 75 refined iterations the proposal failed validation (29 of 75 on cJSON). Few of these failures are the sandbox's import rule (4 of 39 here, 5 of 29 on cJSON); the rest are calls the Hypothesis API does not accept or errors raised while drawing an example.

Why the LLM's generators lower acceptance on parson. Every input parson rejected in either arm is explained by a construct confirmed with a minimal document against the harness. The baseline alternates a grammar-valid document with the same document plus one trailing suffix (a comma, a bracket, a brace, a word, or a NUL byte). cJSON's full-buffer entry point rejects every suffix except NUL, which is what gives it room to improve; parson accepts any bytes after a complete value, so none is rejected, and its only baseline rejections (329 of 7,500) are objects whose key contains an escaped NUL, which parson's source states it does not support. The baseline therefore starts at 95.6% with almost nothing for refinement to fix. The refined generators produce grammar-valid constructs that parson refuses. Of the 1,063 inputs it rejected, 474 contain a duplicate key, 427 an unpaired surrogate escape, 138 a number that overflows a double, 136 a zero followed directly by an exponent (`0e5`), and 10 a NUL in a key (an input can have several); only 6 are not valid under the grammar. JSONTestSuite [1] classifies these constructs. Duplicate keys, `0e5`, and an escaped NUL in a key are documents a parser must accept; unpaired surrogates and out-of-range numbers are left to the implementation, as RFC 8259 [17] permits. 614 of the 1,063 rejected inputs, and all 329 baseline rejections, contain at least one must-accept construct; 443 contain only implementation-defined ones. These rejections are deliberate or known (its source documents the NUL-key limitation, and support for `0e5` was proposed and not merged [23]), so we report them as deviations a conformance suite flags, not as new findings. The refined generators reached exactly the inputs on which parson departs from the standard, and the acceptance proxy scored that as a loss. A further 392 inputs (2.2%) failed to encode and count as not accepted, as in RQ1. Finally, parson's parser returns no error detail, so all 1,392 rejections in both arms share one rejection signature: the proxy tells the proposer how many inputs failed, never why.

C. Exploratory parson pilot

Before API access was available, an exploratory pilot ran five iterations on parson whose proposals were not produced by the loop's LLM proposer: they were written outside the loop, in an interactive session with an AI coding assistant that could read parson's source code. The pilot is therefore neither LLM-proposer-driven nor black-box, and we report it only for what it targeted and found, not as evidence about the method. Table III summarizes its iterations.

TABLE III
PARSON, EXPLORATORY PILOT (NOT PART OF THE STATISTICS): FIVE ITERATIONS OF 500 EXAMPLES EACH, WITH PROPOSALS WRITTEN OUTSIDE THE LOOP.

Iter.	Focus (abridged)	Acc./Rej.	Fingerprints
1	Grammar-seeded generator: recursion, numeric edge cases, duplicate keys, trailing garbage	285 / 215	206/500
2	Wide objects/arrays (up to 4000 keys) to hit rehash/resize paths unreached by iter. 1	341 / 159	273/500
3	Hash-bucket collisions, combined wide+deep structures, surrogate escapes at closing quote	442 / 58	254/500
4	Internal error-cleanup free path: structures valid until the final token	85 / 415	214/500
5	Combines iter. 4's free-path shapes with a sibling-cascade free case	139 / 361	302/500

TABLE IV
CJSON REFINED ARM UNDER THREE FEEDBACK MODES, FOUR USABLE SEEDED TRIALS PER MODE (MEAN OVER TRIALS; SHAPES AND REJECTION SIGNATURES PER 500-INPUT ITERATION). DESCRIPTIVE ONLY: NO TEST IS REPORTED AT n=4.

Mode	Acceptance	Shapes	Rej. signatures
`counts`	92.58%	202.2	2.6
`counts+rejections`	96.42%	203.2	11.9
`full`	95.53%	183.1	12.6

No crash was found in the pilot's 3,000 sanitizer-instrumented executions (500 baseline plus 5×500 pilot iterations), verified by grepping every run's stderr for AddressSanitizer, UndefinedBehaviorSanitizer, and LeakSanitizer text and checking for any non-`accepted`/`rejected` status. A source review of the highest-risk code paths – `parse_utf16`'s raw pointer walk across surrogate pairs, `process_string`'s output-buffer sizing, `json_object_grow_and_rehash`'s linear probing, and, informed specifically by iterations 4–5, `json_object_add`'s duplicate-key failure branch and `parse_object_value`/`parse_array_value`'s missing-bracket failure branch – found that each relies on the harness's own null terminator as a safe, in-bounds stopping condition, and that internal cleanup calls to `json_value_free` are consistent with the harness's own top-level call (an object's key count is never incremented before a duplicate is detected, so cleanup never iterates past the last successfully added slot). We report this as a defensible, evidence-supported explanation for the absence of a crash within this budget, not as proof that none exists: five iterations against one pinned commit, exercising the parse and internal free paths but not parson's public mutation API (`json_object_remove`, `json_array_remove`, and the `_replace_*` family), is a bounded search, and we do not claim more than that bound supports.

D. RQ3: does it matter which counts the LLM sees?

Table IV reports the feedback ablation of Section III: the refined arm's recipe under each of the three feedback modes, five seeded trials (seeds 0–4) per mode. In one trial per mode, all five proposals failed validation, leaving no executed inputs; these three trials are excluded, leaving four usable trials per mode.

All three modes reach a similar, near-ceiling acceptance, and their per-trial rates overlap, which is what the no-feedback control predicts: if the counts do not change acceptance at all, which subset of them is shown should not either. Distinct rejection signatures per iteration are lower with `counts` alone (2.6 against 11.9 and 12.6), but this rests on a single trial, the only `counts` trial with any rejected input. At four trials per mode we report it as a descriptive observation, not an effect.

V. DISCUSSION

A. What the LLM contributes, and what the feedback does not

The cJSON result says something narrower than "the LLM makes fuzzing better", and the control arm makes it precise. Given a grammar, the LLM writes generators whose output is almost always valid, and it does so before it has seen a single campaign result. The counts the loop feeds back – how many inputs were accepted, how many distinct shapes and rejection messages appeared – leave acceptance unchanged. The likely reason is how little they say. The LLM learns that some inputs were rejected, but not which inputs or why, and it never sees its own previous generator, so it has nothing concrete to repair. The one place the feedback helps, fewer failed proposals, is where the prompt does carry concrete text: the previous error message.

The comparison with the static baseline also needs care. The baseline's 58.2% is a design choice, since half its inputs are malformed on purpose to exercise rejection paths, and its valid half alone accepts every input and produces more distinct shapes than the LLM's generators. Near-ceiling acceptance is not itself desirable for fuzzing: six of fifteen refined runs produced no rejected input at all, so they never exercised the accept/reject boundary where parser bugs have historically been found. A loop that is to improve a generator needs feedback with content – the rejected inputs themselves, the parser's messages, or knowledge of where implementations differ – and an objective that values reaching the boundary rather than avoiding it.

B. Interpreting the parson result

The replication shows that the acceptance proxy depends on the target in two ways. First, it needs headroom: a baseline built from trailing-byte mutations is almost fully accepted by a parser that ignores trailing bytes, so there is little left for refinement to raise. Second, it needs the target to be strict in the places the grammar is, and to say why it rejects: parson refuses grammar-valid constructs that cJSON accepts and reports no error detail, so a proposer shown only counts and a single rejection signature cannot tell that its duplicate

keys or `0e5` numbers are the problem. The LLM's generators produce constructs the baseline never does – duplicate keys, surrogate escapes, numbers such as `0e5` – and on this target that costs acceptance, although most of these inputs are ones a conformance suite says a parser must accept. For fuzzing this is not wasted effort, since those inputs exercise parson's rejection paths, but it means acceptance rate cannot serve as the objective on its own, and it motivates a proxy that separates grammar-valid rejections from malformed input.

Finding no crash in 25,500 sanitizer-instrumented executions on parson, and none in the exploratory pilot, is evidence about this pipeline's search within this budget on this pinned commit, not evidence about parson's correctness in general. It cannot distinguish "no bug in the code paths we exercised" from "a bug exists in a code path we did not exercise," such as parson's public mutation API, which the black-box parse-then-free harness used here cannot reach at all.

What the parson runs do establish is narrower and more mechanical: every non-target-specific pipeline component (`runner`, `campaign`, `proposal`, `refinement`, `triage`, `minimize`) worked unmodified against a second parser with a different, and in several respects opposite, set of grammar adaptations, with only a new harness and build script required. The pipeline is portable across targets; the effect of its acceptance proxy is not.

C. Threats to validity

Construct validity. A "crash" in this pipeline is defined purely by sanitizer output and process signals; it cannot detect a parser that silently misparses, accepts input the grammar disallows, or otherwise violates the specification without a memory-safety fault. The grammar mismatches documented in Sections III and IV (duplicate keys, full-buffer consumption) were found by deliberately generating inputs in that region and reading the result, not by any automated oracle; a generator that never targets an under-specified region of the grammar would not surface this class of finding at all.

Internal validity. The parson replication ran 24 days after the cJSON runs, with the harness rebuilt by Homebrew clang 23.1 because a toolchain upgrade had removed the clang 22 sanitizer runtime it was linked to; acceptance depends on parson's parsing logic, not on the sanitizer runtime, and every binary is identified by hash in its run's reproducibility report. The no-feedback arm ran 44 days after the refined arm, on a rebuilt cJSON harness whose replay of all 7,500 seeded baseline inputs matched the original outcomes exactly. Both arms used the model identifier `gpt-4.1-mini`, but a hosted model can change behind a fixed identifier, and the API is not seedable, so we cannot rule out drift between the two dates; the refined arm's own first-iteration generators, written under the original model before any feedback, already reached 96.6–100% acceptance, which points the same way as the control. The structural-fingerprint proxy truncates at 256 bytes (Section III), so it systematically undercounts diversity among longer, deeply nested inputs – exactly the inputs a refinement loop chasing "more diversity" is most likely to produce over successive iterations, which could understate the true diversity signal reflected in the descriptive fingerprint counts we report.

Statistical validity. n=15 runs per arm is modest. The exact tests make no distributional assumption, but $p \approx 1.29 \times 10^{-8}$ is the smallest value fifteen runs per arm can produce, not an effect size, so we report per-run rates (Fig. 1) and Cliff's δ. The control's conclusion is a non-detection: with n=15 it rules out a large effect of feedback on acceptance ($|\delta| = 0.02$), not every small one. The comparison of failed proposals is exploratory and was not part of the pre-registered analysis. Arms execute different numbers of inputs, so only rates are tested, and count-based measures are reported per 500-input iteration.

External validity. All results are on JSON, on two relatively small and independently well-tested C parsers, with one LLM (`gpt-4.1-mini`) at one temperature setting for every LLM-driven arm. The two parsers already disagree on the direction of the effect; we make no claim that either magnitude, or the absence of a crash, generalizes to other formats, other parsers, or other proposer models; in particular, a stronger model might make use of counts that `gpt-4.1-mini` does not.

D. When this approach is, and is not, the right tool

The motivating case for a coverage-free feedback loop is a target that genuinely cannot be instrumented – a vendored dependency pinned at a specific commit for compatibility reasons, a license-restricted or closed-source binary, or a cross-compiled/embedded parser where an instrumented build would not be representative of what ships. Where coverage instrumentation *is* available, a coverage-guided or grammar-aware coverage-guided fuzzer (Section II) has strictly more signal to work with and should be preferred; this pipeline is not proposed as a replacement for that setting, only as an option for the black-box one.

E. Future work

Several extensions are direct given the existing infrastructure. First, the control arm shows that counts alone do not improve the generator, so the natural next experiment gives the feedback content: the rejected inputs themselves, the parser's messages where it has any, the previous generator's source, or knowledge of where implementations of the format differ. Second, an objective that rewards a target reject rate, rather than treating "more accepted" as better without limit, could test whether boundary-probing inputs survive refinement. Third, and most directly relevant to parson, driving `json_value_free` through parson's public mutation API rather than only the parse-then-free path would exercise object lifetime code this black-box harness cannot currently reach at all. Fourth, comparing against an existing external fuzzer under an equalized time or execution budget, rather than only against this project's own static baseline, is the most direct way to situate the magnitude of the cJSON result against prior work.

Two further extensions are larger in scope and deliberately left for future work rather than attempted here. One is broadening beyond a second same-language, same-format

target: a third C JSON parser would add little to what the parson replication already shows about target dependence, whereas a target in a different format, or a memory-safe language where a "crash" oracle must be redefined around uncaught exceptions and API contract violations rather than sanitizer output – for instance, JSON deserialization in a Dart/Flutter mobile application – would test generalization along a materially different axis than implementation alone. The other is applying this pipeline to a live, running mobile or server application rather than a bounded harness process, which would require redesigning the outcome-classification step in Section III around that environment's own failure signals.

VI. Conclusion

We asked what an LLM contributes when it writes and rewrites a grammar-based generator from a formal grammar and black-box feedback alone, with no coverage instrumentation. On cJSON, its generators raised acceptance from a static baseline's 58.2% to 97.1% over fifteen seeded runs. A pre-registered control showed where that gain comes from: with every campaign result withheld, the LLM's generators were statistically indistinguishable ($p \approx 0.95$, $\delta = -0.02$). The model writes a valid generator from the grammar; the counts the loop feeds back do not make it better, and the one thing the feedback measurably helps with is avoiding proposals that fail to run. A pre-registered replication on parson reversed the acceptance effect, and every rejected input was explained: most contain constructs a conformance suite says a parser must accept, so the generators reached exactly where parson departs from the standard, and the acceptance score counted it as failure. No crash was found on either parser.

The lesson for coverage-free fuzzing, which matters for targets coverage-guided fuzzers cannot instrument – pinned vendored dependencies, closed binaries, embedded or cross-compiled parsers – is that cheap black-box counts are not enough to steer an LLM. Useful feedback has to carry content: which inputs failed and why, and where implementations of the same grammar differ. The same grammar admits directly opposite validity decisions on two parsers (duplicate keys accepted by one, rejected by the other), which is exactly the target-specific knowledge a formal specification cannot supply and a refinement loop would need.

We release the full pipeline, both harnesses, every campaign artifact, the control arm's protocol, and the reproducibility tooling described in Section III, so that every comparison in Section IV can be checked, extended, or falsified directly.[1]

Acknowledgment

Use of generative AI. An AI coding assistant, Claude Code [24], was used throughout this work: to implement the harnesses and pipeline, to run and analyze the experiments, and to draft and revise the text of every section. The OpenAI model named in Section III is the system under study, not a writing aid. The work was directed, and all of its content reviewed, by the author(s), with whom full responsibility for it rests.

[1] https://anonymous.4open.science/r/agentic-grammar-fuzzing-6177/

References

[1] N. Seriot, "JSONTestSuite: A comprehensive test suite for RFC 8259 compliant JSON parsers," https://github.com/nst/JSONTestSuite, 2016, companion to the article "Parsing JSON is a Minefield".

[2] M. Zalewski, "American fuzzy lop," https://lcamtuf.coredump.cx/afl/, 2014, accessed 2026.

[3] A. Fioraldi, D. C. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, 2020.

[4] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2021.

[5] P. Purdom, "A sentence generator for testing parsers," *BIT Numerical Mathematics*, vol. 12, no. 3, pp. 366–375, 1972.

[6] J. Wang, B. Chen, L. Wei, and Y. Liu, "Superion: Grammar-aware greybox fuzzing," in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 724–735.

[7] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, "NAUTILUS: Fishing for deep bugs with grammars," in *Proceedings of the 26th Network and Distributed System Security Symposium (NDSS)*, 2019.

[8] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2000, pp. 268–279.

[9] D. R. MacIver, Z. Hatfield-Dodds, and Many other contributors, "Hypothesis: A new approach to property-based testing," *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.

[10] V. Vikram, C. Lemieux, J. Sunshine, and R. Padhye, "Can large language models write good property-based tests?" arXiv preprint arXiv:2307.04346, 2023.

[11] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023, pp. 423–435.

[12] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024.

[13] J. Ackerman and G. Cybenko, "Large language models for fuzzing parsers (registered report)," in *Proceedings of the 2nd International Fuzzing Workshop (FUZZING)*, 2023, pp. 31–38.

[14] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023.

[15] K. Zhang, Z. Li, D. Wu, S. Wang, and X. Xia, "Low-cost and comprehensive non-textual input fuzzing with LLM-synthesized input generators," in *Proceedings of the 34th USENIX Security Symposium*, 2025.

[16] C. Chen, B. Dolan-Gavitt, and Z. Lin, "ELFuzz: Efficient input generation via LLM-driven synthesis over fuzzer space," in *Proceedings of the 34th USENIX Security Symposium*, 2025.

[17] T. Bray, "The JavaScript Object Notation (JSON) data interchange format," RFC 8259, IETF, 2017.

[18] J. Möller, F. Weißberg, L. Pirch, T. Eisenhofer, and K. Rieck, "Cross-language differential testing of JSON parsers," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIA CCS)*, 2024.

[19] D. Gamble, "cJSON: Ultralightweight JSON parser in ANSI C," https://github.com/DaveGamble/cJSON, 2024, version v1.7.19, commit c859b25.

[20] T. Parr, *The Definitive ANTLR 4 Reference*, 2nd ed. Pragmatic Bookshelf, 2013.

[21] "ANTLR grammars-v4: JSON grammar," https://github.com/antlr/grammars-v4/blob/master/json/JSON.g4, 2024.

[22] K. Gabis, “parson: Lightweight JSON library written in C,” https://github.com/kgabis/parson, 2024, version 1.5.3, commit ba29f4e.
[23] “Add compatibility for scientific notation,” parson pull request #188, https://github.com/kgabis/parson/pull/188, 2022.
[24] Anthropic, “Claude code,” https://claude.com/product/claude-code, 2026.